Driver Drowsiness Detection Using CNN

A beginner-friendly Google Colab notebook. It trains a CNN to classify eye images as **Closed** or **Open**, then uses MediaPipe to locate eyes in an uploaded video and the CNN to classify them.

Run cells from top to bottom. The first run downloads the Kaggle dataset. You may need to authorize KaggleHub if prompted.

Note: This is an academic demo, not a certified vehicle-safety system. The video crop style may differ from the training images, so test results carefully.

1. Install and import libraries

```
!pip -q install kagglehub mediapipe==0.10.21
```

```
import os
from pathlib import Path
import numpy as np
import tensorflow as tf
import kagglehub
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt
from google.colab import files
from IPython.display import display, HTML, Audio
import base64

print('TensorFlow:', tf.__version__)
print('GPU:', tf.config.list_physical_devices('GPU'))
```

2. Download and locate the MRL eye dataset

The Kaggle dataset is expected to contain `Close-Eyes` and `Open-Eyes` folders. This cell searches for those folders instead of assuming an exact download path.

```
dataset_download_path = kagglehub.dataset_download('imadeddinedjerarda/mrl-e
print('Downloaded to:', dataset_download_path)

dataset_path = None
for folder, subfolders, filenames in os.walk(dataset_download_path):
    names = {name.lower(): name for name in subfolders}
    if 'close-eyes' in names and 'open-eyes' in names:
        dataset_path = folder
        break

if dataset_path is None:
    raise FileNotFoundError('Could not find Close-Eyes and Open-Eyes folders
```

```
closed_folder = os.path.join(dataset_path, 'Close-Eyes')
open_folder = os.path.join(dataset_path, 'Open-Eyes')
print('Dataset folder:', dataset_path)
print('Closed-eye files:', len(os.listdir(closed_folder)))
print('Open-eye files:', len(os.listdir(open_folder)))
```

3. Load training and validation images

Keras reads the two folders as binary classes. Label `0` is Closed and label `1` is Open. The 80/20 split here is for training and validation; it is not a subject-independent research evaluation.

```
IMAGE_SIZE = (64, 64)
BATCH_SIZE = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    class_names=['Close-Eyes', 'Open-Eyes'],
    validation_split=0.2,
    subset='training',
    seed=42,
    image_size=IMAGE_SIZE,
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    label_mode='binary'
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    class_names=['Close-Eyes', 'Open-Eyes'],
    validation_split=0.2,
    subset='validation',
    seed=42,
    image_size=IMAGE_SIZE,
    color_mode='grayscale',
    batch_size=BATCH_SIZE,
    label_mode='binary'
)

print('Class order:', train_ds.class_names)
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

4. View a few training images

```
plt.figure(figsize=(8, 5))
for images, labels in train_ds.take(1):
    for i in range(min(8, len(images))):
        plt.subplot(2, 4, i + 1)
        plt.imshow(images[i].numpy().squeeze(), cmap='gray')
        plt.title('Open' if int(labels[i].numpy()[0]) == 1 else 'Closed')
        plt.axis('off')
```

```
    plt.tight_layout()
    plt.show()
```

5. Build and train the CNN

Convolution layers learn image patterns, pooling reduces feature-map size, and the sigmoid output predicts Open-eye probability.

```
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(64, 64, 1)),
        tf.keras.layers.Rescaling(1./255),
        tf.keras.layers.Conv2D(16, 3, activation='relu'),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation='relu'),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])

    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accura
    model.summary()

    early_stop = tf.keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=2, restore_best_weights=True
    )

    history = model.fit(
        train_ds, validation_data=val_ds,
        epochs=6, callbacks=[early_stop]
    )
```

6. Show accuracy and loss

```
    plt.figure(figsize=(10, 4))
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Training')
    plt.plot(history.history['val_accuracy'], label='Validation')
    plt.title('Accuracy'); plt.xlabel('Epoch'); plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Training')
    plt.plot(history.history['val_loss'], label='Validation')
    plt.title('Loss'); plt.xlabel('Epoch'); plt.legend()
    plt.tight_layout(); plt.show()

    val_loss, val_accuracy = model.evaluate(val_ds)
    print(f'Validation accuracy: {val_accuracy * 100:.2f}%')
```

7. Save the trained CNN model

```
MODEL_PATH = '/content/drowsiness_cnn.keras'
model.save(MODEL_PATH)
print('Saved:', MODEL_PATH)
```

8. Set up MediaPipe and eye-cropping function

MediaPipe locates eye landmarks. The CNN—not EAR—classifies each cropped eye as Open or Closed.

```
face_mesh = mp.solutions.face_mesh.FaceMesh(
    static_image_mode=False,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

LEFT_EYE = [362, 385, 387, 263, 373, 380]
RIGHT_EYE = [33, 160, 158, 133, 153, 144]

def crop_eye(frame, landmarks, indices):
    h, w = frame.shape[:2]
    points = [(int(landmarks[i].x * w), int(landmarks[i].y * h)) for i in ir
    x1, x2 = min(p[0] for p in points), max(p[0] for p in points)
    y1, y2 = min(p[1] for p in points), max(p[1] for p in points)
    pad_x = max(4, int((x2 - x1) * 0.7))
    pad_y = max(4, int((y2 - y1) * 1.2))
    x1, x2 = max(0, x1-pad_x), min(w, x2+pad_x)
    y1, y2 = max(0, y1-pad_y), min(h, y2+pad_y)
    crop = frame[y1:y2, x1:x2]
    if crop.size == 0:
        return None, None
    crop = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
    crop = cv2.resize(crop, (64, 64))
    return crop, (x1, y1, x2, y2)

print('MediaPipe is ready.')
```

9. Upload a short face video and process it

Use a short, front-facing video with clear lighting. For a demonstration, include a period with eyes open and a deliberate eye closure. Do not record or test while driving.

```
uploaded = files.upload()
video_path = next(iter(uploaded))

cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS)
if not fps or fps <= 0:
    fps = 25
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

```
if width == 0 or height == 0:
    raise ValueError('Could not read the uploaded video. Try an MP4 or AVI f

output_path = '/content/drowsiness_output.mp4'
writer = cv2.VideoWriter(output_path, cv2.VideoWriter_fourcc(*'mp4v'), fps,
closed_frames = 0
alert_frames = 0
CLOSED_SECONDS = 0.8
frame_limit = max(1, int(fps * CLOSED_SECONDS))

while True:
    ok, frame = cap.read()
    if not ok:
        break

    result = face_mesh.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    status = 'NO FACE DETECTED'
    color = (0, 165, 255)

    if result.multi_face_landmarks:
        landmarks = result.multi_face_landmarks[0].landmark
        left, left_box = crop_eye(frame, landmarks, LEFT_EYE)
        right, right_box = crop_eye(frame, landmarks, RIGHT_EYE)

        if left is not None and right is not None:
            batch = np.array([left, right], dtype='float32').reshape(2, 64,
            probabilities = model.predict(batch, verbose=0).ravel()
            both_closed = (probabilities[0] < 0.5) and (probabilities[1] < 0
            closed_frames = closed_frames + 1 if both_closed else 0

            status = 'EYES CLOSED' if both_closed else 'EYES OPEN'
            color = (0, 200, 255) if both_closed else (0, 255, 0)

            for box in [left_box, right_box]:
                x1, y1, x2, y2 = box
                cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

            if closed_frames >= frame_limit:
                status = 'DROWSINESS ALERT!'
                color = (0, 0, 255)
                alert_frames += 1
        else:
            closed_frames = 0
    else:
        closed_frames = 0

    cv2.putText(frame, status, (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.9, colo
    writer.write(frame)

cap.release()
writer.release()
face_mesh.close()
print('Video processing finished.')
print('Alert frames:', alert_frames)
```

10. Play the processed video and warning sound

```
!ffmpeg -y -loglevel error -i /content/drowsiness_output.mp4 -vcodec libx2

video_bytes = open('/content/final_output.mp4', 'rb').read()
```

```
    video_base64 = base64.b64encode(video_bytes).decode()
    display(HTML(f'<video width="640" controls><source src="data:video/mp4;bas

    if alert_frames > 0:
        sample_rate = 22050
        t = np.linspace(0, 1, sample_rate, endpoint=False)
        tone = 0.4 * np.sin(2 * np.pi * 850 * t)
        display(Audio(tone, rate=sample_rate))
        print('Drowsiness warning detected.')
    else:
        print('No drowsiness alert detected in this video.')

    files.download('/content/final_output.mp4')
```

Viva quick notes

- **CNN:** A neural network commonly used for image classification.
- **Convolution:** Learns visual features such as edges and patterns.
- **Max pooling:** Reduces feature-map size.
- **MediaPipe:** Finds facial landmarks and helps locate the eyes.
- **OpenCV:** Reads and processes video frames.
- **Sigmoid:** Produces a value between 0 and 1 for binary classification.
- **Drowsiness rule:** In this demo, both eyes must be classified as closed for about 0.8 seconds before an alert appears.

Limitations: The MRL dataset images may look different from eye crops taken from a full-face video. Validate the video predictions on multiple people and lighting conditions. This is a classroom prototype, not a certified driver-safety system. The frame-level random validation split may also contain images from the same participant in both subsets.